Table of Contents

Chapter 1: Introduction

1.1 Project Overview

HostelHub is a production-quality, web-based smart hostel management system designed to digitize and streamline hostel operations including room allocation, student administration, financial tracking, complaint management, leave processing, and security monitoring.

The application is built using a full-stack JavaScript approach — React.js on the client-side and Node.js with Express.js on the server-side — backed by a normalized MySQL relational database containing 18 interdependent tables.

The system supports five distinct user roles: Administrator, Warden, Student, Security Guard, and Housekeeper, each with a dedicated dashboard, access-controlled API routes, and tailored feature set. All communication between the browser and server happens over secure, stateless REST API calls authenticated using JSON Web Tokens (JWT), while all user passwords are stored as irreversible cryptographic hashes using Bcrypt.js.

The frontend consists of 21 React page components organized under a Single Page Application (SPA) architecture using React Router DOM v6. The backend exposes 17 modular route files covering all domains of the application — from authentication and room management to broadcasting alerts and processing gate logs.

1.2 Purpose and Objectives

The primary purpose of HostelHub is to replace fragmented, manual paper-based hostel administration with a centralized, transactional, and auditable digital system.

Core Objectives:

- Real-Time Room Tracking: Dynamically track room capacity vs. occupancy to prevent double bookings through database-level constraints.
- Transparent Financial Workflows: Allow students to submit fee payment proofs with bank transaction IDs for warden/admin audit and approval.
- Structured Grievance Redressal: Digital complaint ticketing system categorized by type and urgency level.
- Secure Gate & Attendance Auditing: Log student entry/exit through approved gate logs with security guard ID and timestamp.
- Role-Based Access Control (RBAC): Each user role can only view and operate on data relevant to their authorization level.

1.3 Scope of the System

- Transactional Integrity: Multi-step operations are atomic database transactions under MySQL's ACID guarantees.
- RBAC Enforcement: JWT payload carries the user's role field, validated by Express.js middleware on every protected endpoint.

- Referential Integrity: Foreign key constraints with ON DELETE CASCADE and ON DELETE SET NULL prevent orphaned records.
- Data Domain Enforcement: ENUM data types across 12+ columns guarantee only valid categorical values are stored.

1.4 Introduction to Core Technologies Used

1.4.1 HTML5

HTML5 forms the structural skeleton of the React component templates. Semantic elements such as <header>, <nav>, <main>, <section>, and <footer> organize the dashboard layout meaningfully. Built-in HTML5 validation attributes (required, type='email', minLength, pattern) provide client-side sanity checks before data is dispatched to the backend.

1.4.2 CSS3 & Tailwind CSS (v3.3.3)

Tailwind CSS is a utility-first CSS framework that eliminates the need for writing external .css files by providing pre-built class names applied directly in JSX markup. The project uses Tailwind alongside PostCSS and Autoprefixer.

- Responsive Breakpoints: sm:, md:, lg:, xl: classes ensure the dashboard adapts from mobile to widescreen displays.
- Dark Mode Aesthetics: Dark background gradients with high-contrast text.
- Glassmorphism: Semi-transparent cards with backdrop-blur utilities.
- Micro-animations: Smooth transition, hover:scale-105, and duration-300 effects.

1.4.3 JavaScript (ES6+) & React.js (v18.2.0)

JavaScript ES6+ is the primary programming language of the entire project — both on the frontend (browser) and backend (Node.js runtime).

- React.js: Declarative, component-based UI library using a Virtual DOM. Built with Vite (v4.4.5) for Hot Module Replacement (HMR).
- React Router DOM (v6.16.0): Client-side routing across 21 pages without full page reloads. Protected routes redirect unauthenticated users.
- Framer Motion (v10.16.4): Production-grade animation library for page transitions and interactive hover effects.
- React Icons (v4.11.0): Comprehensive SVG icon library (Font Awesome, Material, Feather) used throughout the UI.
- QRCode.react (v4.2.0): Generates QR codes dynamically for student gate passes and security scanning workflows.

1.4.4 Axios (v1.5.0)

Promise-based HTTP client configured with a base URL and an interceptor that automatically attaches the JWT Authorization: Bearer <token> header to every outgoing request after login.

1.4.5 Node.js & Express.js (v4.18.2)

- Routing: 17 distinct route files map HTTP method + URL path combinations to dedicated controller functions.
- Middleware Stack: helmet (v7.0.0), cors (v2.8.5), morgan (v1.10.0), and custom JWT verification middleware.
- File Uploads: multer (v2.1.1) handles multipart form submissions for payment proof image uploads.
- Nodemon (v3.0.1): Auto-restarts the server on file changes during development.

1.4.6 Sequelize ORM (v6.33.0) & mysql2 (v3.6.1)

- mysql2: High-performance Node.js MySQL driver with Promise support and connection pooling.
- Sequelize: ORM providing model-based abstractions over MySQL tables for structured queries.

1.4.7 Security — JWT & Bcrypt.js

- JSON Web Tokens (jsonwebtoken v9.0.2): Signs a JWT containing user id, name, email, and role after successful login.
- Bcrypt.js (v2.4.3): Passes passwords through bcrypt.hash() with a configurable salt round before storage. Login uses bcrypt.compare() without ever decrypting.
- dotenv (v16.3.1): Loads environment variables (DB host, JWT secret) from .env files.

1.4.8 MySQL (v8.0)

- ACID Transactions: Atomicity, Consistency, Isolation, Durability for multi-step operations.
- Referential Integrity: Foreign Key constraints with cascading delete rules.
- Optimized Indexing: INT AUTO_INCREMENT Primary Keys on all 18 tables.
- Strict Typing: VARCHAR, INT, TEXT, ENUM, DECIMAL, DATE, TIMESTAMP, BOOLEAN column definitions.

Chapter 2: System Requirement Specifications

2.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3/i5/i7 or AMD equivalent (2.0 GHz+)
RAM	8 GB minimum; 16 GB recommended
Storage	5 GB free disk space
Client Device	Any modern smartphone, laptop, tablet, or desktop
Display	Minimum 1024×768 pixels resolution
Network	Broadband internet or local LAN connection

2.2 Software Requirements

Category	Tool / Version
Operating System	Windows 10/11, macOS 12+, or Ubuntu Linux 20.04+
Database Server	MySQL Server v8.0 or MariaDB v10.4+
Runtime Environment	Node.js v16.x or newer; NPM v8.x+
Web Browser	Google Chrome v100+, Firefox v100+, Edge, or Safari
Code Editor	Antigravity IDE
Database Client	MySQL Workbench or phpMyAdmin
Version Control	Git v2.x
Build Tool	Vite v4.4.5 (frontend), Nodemon v3.0.1 (backend)

2.3 NPM Packages & Dependencies

Frontend Dependencies (frontend/package.json)

Package	Version	Purpose
react	^18.2.0	Core UI component library
react-dom	^18.2.0	React DOM rendering engine
react-router-dom	^6.16.0	Client-side SPA routing & protected routes
axios	^1.5.0	Asynchronous HTTP REST API client
qrcode.react	^4.2.0	QR code generator for gate pass scanning
tailwindcss	^3.3.3	Utility-first CSS styling framework
vite	^4.4.5	Fast frontend build tool with HMR
autoprefixer	^10.4.16	PostCSS CSS vendor prefix plugin
postcss	^8.4.31	CSS transformation toolchain

Backend Dependencies (backend/package.json)

Package	Version	Purpose
express	^4.18.2	RESTful API web framework for Node.js
mysql2	^3.6.1	High-performance MySQL database driver
jsonwebtoken	^9.0.2	JWT creation, signing, and verification
bcryptjs	^2.4.3	Password hashing with salting
cors	^2.8.5	Cross-Origin Resource Sharing middleware
morgan	^1.10.0	HTTP request logger for development
nodemon	^3.0.1	Auto-restart server on file changes (dev)

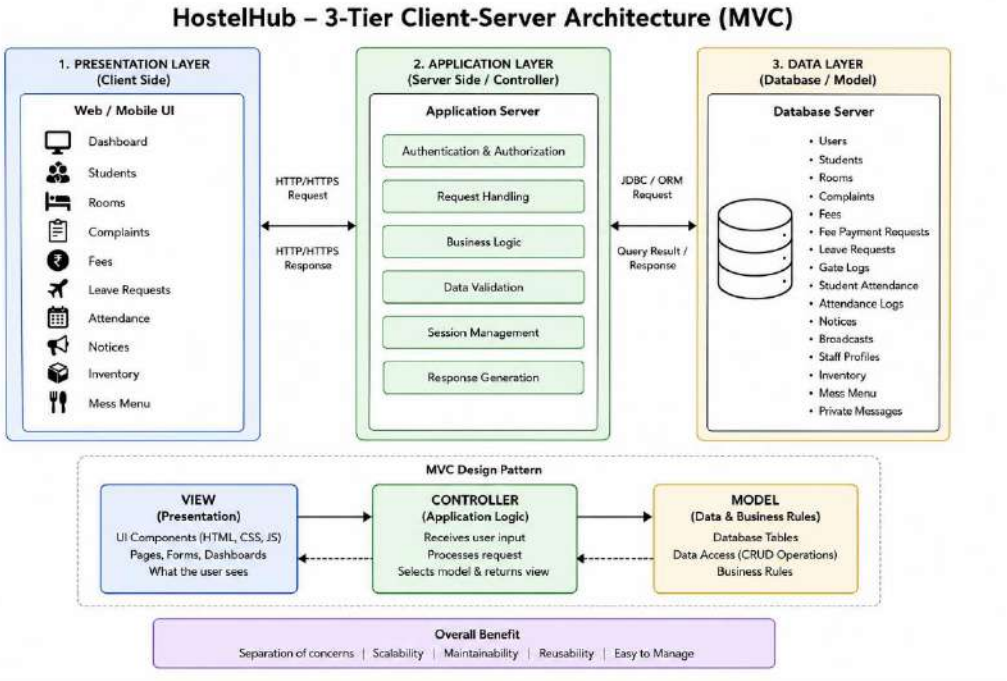
Chapter 3: System Analysis & Design

3.1 Existing System vs. Proposed System

Feature	Existing System	Proposed HostelHub System
Room Allocation	Manual paper registers, prone to overbooking.	Real-time constraint: occupied < capacity validated before allocation.
Complaint Logging	Written slips, frequently lost, no tracking.	Digital ticketing with ENUM categories and priority levels.
Fee Management	Manual bank receipt checks, hand-calculated dues.	Digital payment proof uploads with DECIMAL(10,2) precision.
Security Logs	Paper logbooks, incomplete or illegible.	Timestamped gate logs linked to student USN and guard ID.
Leave Approvals	Handwritten apps, delayed by warden absence.	Digital portal; wardens approve with a single click.
Attendance	Manual roll calls, no digital history.	Daily digital attendance with unique composite key constraint.
Notice Board	Physical pin-up boards, stale notices not removed.	Digital board with timestamps; archived and paginated.
Messaging	In-person or phone calls between students and wardens.	Private messaging with is_read flag and role-based routing.

3.2 System Architecture (MVC & Client-Server)

HostelHub is built on a 3-tier Client-Server architecture structured around the Model-View-Controller (MVC) design:



3.3 REST API Routes Overview

Route Prefix	Route File	Responsibility
/api/auth	authRoutes.js	Login, register, token validation
/api/students	studentRoutes.js	Student CRUD, profile, roommates
/api/rooms	roomRoutes.js	Room listing, allocation, status updates
/api/complaints	complaintRoutes.js	File, view, and resolve complaints
/api/fees	feeRoutes.js	Invoice generation, payment proof upload
/api/leaves	leaveRoutes.js	Leave submission and approval workflow
/api/analytics	analyticsRoutes.js	Dashboard KPI metrics and statistics
/api/notices	noticeRoutes.js	Create, read, and delete notice posts
/api/inventory	inventoryRoutes.js	Track hostel assets and stock levels
/api/mess-menu	messMenuRoutes.js	View and update weekly mess schedule
/api/admin	adminRoutes.js	Admin-only: user management, system config
/api/broadcasts	broadcastRoutes.js	Emergency alert broadcasting
/api/security	securityRoutes.js	Shift assignments, alerts, incident reports
/api/messages	messageRoutes.js	Private messaging between users
/api/staff	staffRoutes.js	Staff profile management
/api/attendance	attendanceRoutes.js	Staff check-in/out, student attendance
/api/gate	gateRoutes.js	Gate entry/exit logging and QR scanning

3.4 Database Relational Schema — All 16 Tables

The HostelHub database schema consists of 16 relational tables that together model every aspect of hostel operations.

Table 1: users

Central identity and authentication table for all system actors.

Column	Type	Constraint	Description
Id	INT	PK, AUTO_INCREMENT	Unique user identifier
Name	VARCHAR(100)	NOT NULL	Full display name
Email	VARCHAR(100)	UNIQUE, NOT NULL	Login email address
Password	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
Role	ENUM(...)	DEFAULT 'student'	Access level

Table 2: students

Academic profile and hostel enrollment data for registered students.

Column	Type	Constraint	Description
student_id	INT	PK, AUTO_INCREMENT	Unique student ID
user_id	INT	FK → users.id CASCADE	Link to login credentials
course	VARCHAR(100)	—	Degree program (e.g., B.E., MCA)
branch	VARCHAR(100)	—	Department (e.g., CSE, ECE)
year	INT	—	Academic year (1–4)
phone	VARCHAR(15)	—	Contact number
blood_group	VARCHAR(5)	—	Student blood group
address	TEXT	—	Permanent residential address
room_id	INT	FK → rooms.room_id SET NULL	Currently allocated room
usn	VARCHAR(20)	UNIQUE	University Seat Number
semester	INT	—	Current semester
status	ENUM(...)	DEFAULT 'active'	Enrollment status

Table 3: rooms

Physical hostel room inventory with real-time occupancy tracking.

Column	Type	Constraint	Description
room_id	INT	PK, AUTO_INCREMENT	Unique room identifier
room_number	VARCHAR(10)	UNIQUE, NOT NULL	Room label (e.g., 'A-101')
block	VARCHAR(10)	—	Hostel block (e.g., 'A', 'B')
floor	INT	—	Floor number
capacity	INT	DEFAULT 3	Maximum bed count
occupied	INT	DEFAULT 0	Currently occupied beds
status	ENUM(...)	DEFAULT 'available'	Room viability status

Table 4: complaints

Grievance ticketing system with category and priority classification.

Column	Type	Constraint	Description
complaint_id	INT	PK, AUTO_INCREMENT	Unique ticket ID
student_id	INT	FK → students CASCADE	Reporting student
title	VARCHAR(255)	NOT NULL	Brief complaint summary
description	TEXT	NOT NULL	Full details of the complaint
category	ENUM(...)	DEFAULT 'other'	Complaint type
priority	ENUM(...)	DEFAULT 'medium'	Urgency level
status	ENUM(...)	DEFAULT 'pending'	Resolution stage
created_at	TIMESTAMP	DEFAULT NOW()	Filing timestamp

Table 5: fees

Financial invoice records issued to students.

Column	Type	Constraint	Description
fee_id	INT	PK, AUTO_INCREMENT	Unique invoice ID
student_id	INT	FK → students CASCADE	Invoice recipient
amount	DECIMAL(10,2)	NOT NULL	Outstanding balance
due_date	DATE	NOT NULL	Payment deadline
status	ENUM(...)	DEFAULT 'unpaid'	Payment state
payment_date	TIMESTAMP	NULL	When payment was confirmed

Table 6: leave_requests

Digital leave application workflow for hostel students.

Column	Type	Constraint	Description
leave_id	INT	PK, AUTO_INCREMENT	Unique leave request ID
student_id	INT	FK → students CASCADE	Applicant student
reason	TEXT	NOT NULL	Detailed reason for leave
destination	VARCHAR(255)	—	Travel destination
from_date	DATE	NOT NULL	Leave start date
to_date	DATE	NOT NULL	Expected return date

Column	Type	Constraint	Description
status	ENUM(...)	DEFAULT 'pending'	Approval state
created_at	TIMESTAMP	DEFAULT NOW()	Application timestamp

Table 7: fee_payment_requests

Digital payment proof submissions uploaded by students for verification.

Column	Type	Constraint	Description
request_id	INT	PK, AUTO_INCREMENT	Unique reference ID
fee_id	INT	FK → fees CASCADE	Target invoice
student_id	INT	FK → students CASCADE	Submitting student
amount	DECIMAL(10,2)	NOT NULL	Amount transferred
transaction_id	VARCHAR(100)	NOT NULL	Bank transaction reference
status	ENUM(...)	DEFAULT 'pending'	Audit state
remarks	VARCHAR(255)	NULL	Auditor remarks
created_at	TIMESTAMP	DEFAULT NOW()	Submission timestamp

Table 8: notices

Administrative and warden announcements displayed on the notice board.

Column	Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Bulletin ID
title	VARCHAR(255)	NOT NULL	Notice heading
content	TEXT	NOT NULL	Full notice body text
created_by	INT	FK → users CASCADE	Admin or Warden user ID
created_at	TIMESTAMP	DEFAULT NOW()	Publication timestamp

Table 9: inventory

Hostel asset and furniture stock tracker.

Column	Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Item ID
name	VARCHAR(100)	NOT NULL	Item name (e.g., 'Mattress')
quantity	INT	DEFAULT 0	Available unit count
description	TEXT	—	Detailed item description
last_updated	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Auto update tracking

Table 10: mess_menu

Weekly dietary schedule for the hostel mess.

Column	Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Day ID
day_name	ENUM(Mon–Sun)	UNIQUE, NOT NULL	Day of week
tiffin	TEXT	—	Breakfast/tiffin menu
lunch	TEXT	—	Lunch menu
snacks	TEXT	—	Evening snacks
dinner	TEXT	—	Dinner menu

Table 11: private_messages

Direct messaging channel between students, wardens, and admins.

Column	Type	Constraint	Description
message_id	INT	PK, AUTO_INCREMENT	Message ID
sender_id	INT	FK → users CASCADE	Sending user
recipient_id	INT	FK → users SET NULL	Recipient user
recipient_role	ENUM(...)	NULL	Role-based filtering
content	TEXT	NOT NULL	Message body
is_read	BOOLEAN	DEFAULT FALSE	Read/unread flag
created_at	TIMESTAMP	DEFAULT NOW()	Send timestamp

Table 12: broadcasts

System-wide alert and emergency broadcasts published by administrators.

Column	Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Broadcast ID
message	TEXT	NOT NULL	Alert content
type	ENUM(...)	DEFAULT 'alert'	Severity level
is_active	BOOLEAN	DEFAULT TRUE	Active/dismissed status
created_by	INT	FK → users CASCADE	Publishing user
created_at	TIMESTAMP	DEFAULT NOW()	Broadcast timestamp

Table 13: staff_profiles

Extended operational profiles for all non-student employees.

Column	Type	Constraint	Description
profile_id	INT	PK, AUTO_INCREMENT	Profile ID

Column	Type	Constraint	Description
user_id	INT	FK → users CASCADE	Linked login account
Phone	VARCHAR(15)	—	Office contact number
Address	TEXT	—	Staff residential address
designation	VARCHAR(100)	—	Job title
experience	VARCHAR(50)	—	Years of service
blood_group	VARCHAR(5)	—	Staff blood group
emergency_contact	VARCHAR(15)	—	Emergency contact number

Table 14: gate_logs

Timestamped record of all student entry and exit events at the hostel gate.

Column	Type	Constraint	Description
log_id	INT	PK, AUTO_INCREMENT	Gate log ticket ID
student_id	INT	FK → students CASCADE	Identified student
type	ENUM('entry','exit')	NOT NULL	Movement direction
destination	VARCHAR(255)	—	Purpose/destination
timestamp	TIMESTAMP	DEFAULT NOW()	Event time
security_id	INT	FK → users CASCADE	Authorizing security guard

Table 15: attendance_logs

Staff shift check-in and check-out time recorder.

Column	Type	Constraint	Description
attendance_id	INT	PK, AUTO_INCREMENT	Log ID
user_id	INT	FK → users CASCADE	Staff member
type	ENUM('check-in','check-out')	NOT NULL	Shift action type
timestamp	TIMESTAMP	DEFAULT NOW()	Exact shift timestamp

Table 16: student_attendance

Daily student attendance register maintained by wardens.

Column	Type	Constraint	Description
attendance_id	INT	PK, AUTO_INCREMENT	Attendance log ID
student_id	INT	FK → students CASCADE	Tracked student

Column	Type	Constraint	Description
status	ENUM(...)	NOT NULL	Attendance status
date	DATE	NOT NULL	Calendar date
marked_by	INT	FK → users CASCADE	Warden who recorded it
created_at	TIMESTAMP	DEFAULT NOW()	Entry creation timestamp
UNIQUE KEY	(student_id, date)	COMPOSITE	Prevents duplicate per day

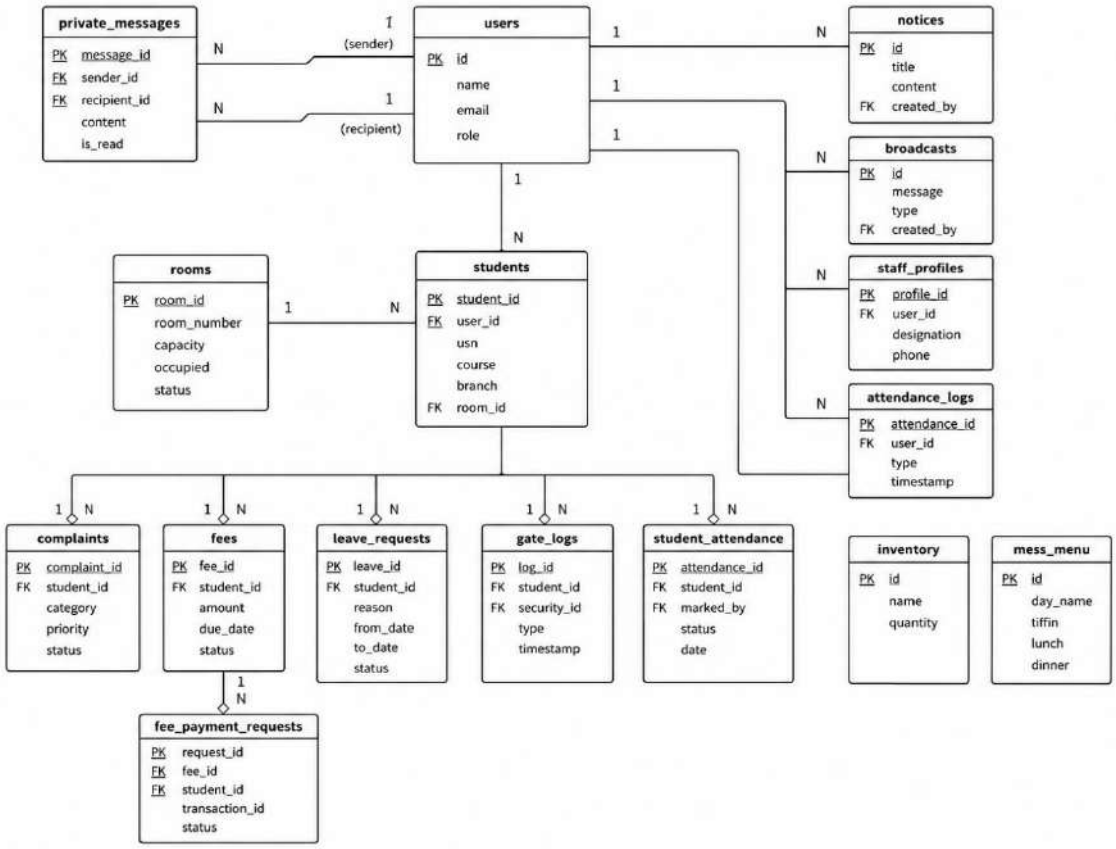
3.5 ER Design Concepts & Constraints

Entity Relationship Overview

1. **Users Entity**
   The **Users** entity acts as the central table in the Hostel Management System database. It stores common information such as user ID, name, email, and role. Different users such as students and staff are managed through this entity.
2. **Students Entity**
   The **Students** entity stores student-specific details such as USN, course, branch, and room allocation. Each student is linked to a user account and connected with other hostel-related records.
3. **Rooms Entity**
   The **Rooms** entity contains details such as room number, capacity, occupancy, and room status. It helps in managing hostel room allocation for students.
4. **Complaints Management**
   The **Complaints** entity stores complaints raised by students. It includes complaint category, priority level, and complaint status for tracking and resolution.
5. **Fee Management**
   The **Fees** and **Fee Payment Requests** entities manage fee records, due dates, payment amounts, and transaction request details of students.
6. **Leave Management**
   The **Leave Requests** entity records student leave applications including reason, from-date, to-date, and approval status.
7. **Attendance Management**
   The **Student Attendance** and **Attendance Logs** entities are used to maintain attendance records of students and staff for monitoring hostel activities.
8. **Communication Module**
   The **Notices** and **Broadcasts** entities are used to share announcements, important messages, and hostel-related updates with students and staff.
9. **Additional Management Modules**
   The **Inventory** entity manages hostel resources and item quantities, while the **Mess Menu** entity stores daily meal schedules such as tiffin, lunch, and dinner.

10. **Overall ER Design Purpose**

The ER design provides a structured representation of all entities and their relationships. It helps in organizing hostel data efficiently, reducing redundancy, and ensuring consistency across all hostel management operations.



Chapter 4: System Implementation & Modules

4.1 Student Module

Frontend Pages: Login.jsx, Profile.jsx, FeeList.jsx, LeaveList.jsx, ComplaintList.jsx, Notices.jsx, MessMenu.jsx, StudentSupport.jsx, StudentNetwork.jsx

- Profile Dashboard (Profile.jsx): Displays USN, course, branch, academic year, semester, contact details, blood group, and current room number with roommate listing.
- Fee Management (FeeList.jsx): Lists all fee invoices with due dates and statuses. Students upload payment proof images and bank transaction IDs for warden verification.
- Leave Portal (LeaveList.jsx): Students submit leave requests with reason, destination, and date range. Real-time status tracking (pending/approved/rejected).
- Complaint Register (ComplaintList.jsx): File maintenance complaints selecting category and priority. Resolution progress tracked live.
- Notice Board (Notices.jsx): All active administrative announcements in reverse-chronological order.
- Mess Menu (MessMenu.jsx): Current week's full meal schedule (tiffin, lunch, snacks, dinner).
- StudentSupport & StudentNetwork: Private messaging to wardens and fellow student directory.

4.2 Warden / Staff Module

Frontend Pages: Dashboard.jsx, RoomList.jsx, LeaveList.jsx, ComplaintList.jsx, StudentAttendance.jsx, FeeList.jsx, Notices.jsx

- Analytics Dashboard (Dashboard.jsx): Real-time KPIs — total students, occupied beds, pending leave requests, unresolved complaints, outstanding fees, today's attendance.
- Room Registry (RoomList.jsx): Visual table of all rooms. Wardens update room status and reassign allocations.
- Leave Approval (LeaveList.jsx): Approve or reject pending leave applications with a single button click.
- Complaint Processing: Update complaint status (pending → in-progress → resolved) and record warden remarks.
- Daily Attendance (StudentAttendance.jsx): Mark each student present, absent, or on_leave with composite key protection.
- Fee Audit (FeeList.jsx): Review and approve/reject student payment proof submissions.

4.3 Administrator Module

Frontend Pages: Dashboard.jsx, StudentList.jsx, StaffList.jsx, RoomList.jsx, FeeList.jsx, Inventory.jsx, Notices.jsx, AdminMessages.jsx

- System Administration (StudentList.jsx, StaffList.jsx): Full CRUD on all students and staff. Create accounts with auto-generated credentials.
- User Role Management: Register and assign roles (Warden, Security Guard, Housekeeper) via /api/admin.
- Global Notices & Broadcasts: Post notice bulletins and send emergency alerts (alert/emergency/info) system-wide.
- Financial Controls (FeeList.jsx): Issue fee requirements, review and approve payment verification submissions.
- Resource Inventory (Inventory.jsx): Track hostel assets (chairs, tables, mattresses, fans) and update stock quantities.
- Admin Messaging (AdminMessages.jsx): Monitor and participate in the private messaging system across all roles.

4.6 Frontend Page Components Overview

Component File	Role Access	Functionality
Login.jsx	All	User login with JWT issuance
Register.jsx	Admin	New user account creation
Dashboard.jsx	Admin, Warden	Real-time analytics dashboard
Profile.jsx	Student	Student profile and room info
StudentList.jsx	Admin, Warden	Full student directory with CRUD
RoomList.jsx	Admin, Warden	Room registry and allocation
ComplaintList.jsx	Student, Warden	Complaint filing and resolution
FeeList.jsx	Admin, Warden, Student	Invoices and payment management
LeaveList.jsx	Admin, Warden, Student	Leave applications and approvals
StudentAttendance.jsx	Warden	Daily student attendance marking
Notices.jsx	All	Notice board management
MessMenu.jsx	Student, Warden	Weekly mess menu display
Inventory.jsx	Admin	Hostel asset stock management
AdminMessages.jsx	Admin	System-wide message monitoring
StaffList.jsx	Admin	Staff directory and management
StaffProfile.jsx	Warden, Security	Personal staff profile page
SecurityDashboard.jsx	Security	Guard shift and duty panel
StudentSupport.jsx	Student	Private messaging to warden/admin
StudentNetwork.jsx	Student	Student community directory

Chapter 5: Key SQL Queries & DBMS Execution

The following SQL statements cover all standard database verification categories for a DBMS laboratory examination. All queries are optimized for MySQL v8.0 execution.

5.1 System Inspection & Auditing Queries

Query 1: List All Tables in the Database

```
SHOW TABLES;
```

Returns: All 18 table names in the hostelhub database.

Query 2: Inspect Column Structure of Core Tables

```
DESCRIBE users;
DESCRIBE students;
DESCRIBE rooms;
DESCRIBE fees;
```

Shows: Column name, data type, key constraints, default values, and nullable flags.

Query 3: View All Registered Users with Their Roles

```
SELECT id, name, email, role, created_at
FROM users
ORDER BY created_at DESC;
```

5.2 Core Join Queries

Query 4: Fetch Complete Student Profile (Multi-table JOIN)

```
SELECT s.student_id, u.name, u.email, s.usn, s.course,
      s.branch, s.year, s.semester, s.phone, s.status
FROM students s
JOIN users u ON s.user_id = u.id
ORDER BY u.name;
```

Query 5: List All Roommates in a Specific Room (3-Table JOIN)

```
SELECT u.name, s.usn, s.course, s.branch, r.room_number, r.block
FROM students s
JOIN users u ON s.user_id = u.id
JOIN rooms r ON s.room_id = r.room_id
WHERE r.room_number = '101';
```

Query 6: View All Outstanding or Partial Fee Invoices

```
SELECT f.fee_id, u.name, s.usn, f.amount, f.due_date, f.status
FROM fees f
JOIN students s ON f.student_id = s.student_id
JOIN users u ON s.user_id = u.id
WHERE f.status IN ('unpaid', 'partial')
ORDER BY f.due_date ASC;
```

Query 7: View Pending Leave Applications with Student Details

```
SELECT lr.leave_id, u.name, s.usn, lr.reason,
      lr.destination, lr.from_date, lr.to_date, lr.status
FROM leave_requests lr
JOIN students s ON lr.student_id = s.student_id
JOIN users u ON s.user_id = u.id
WHERE lr.status = 'pending'
ORDER BY lr.created_at ASC;
```

5.3 Aggregation & Dashboard Analytics Queries

Query 8: Count Total Active Students

```
SELECT COUNT(*) AS total_students
FROM students
WHERE status = 'active';
```

Query 9: Today's Attendance Breakdown

```
SELECT status, COUNT(*) AS student_count
FROM student_attendance
WHERE date = CURDATE()
GROUP BY status;
```

Query 10: Complaint Summary Grouped by Status

```
SELECT status, COUNT(*) AS complaint_count
FROM complaints
GROUP BY status;
```

Query 11: Count Unique Students Who Filed Complaints

```
SELECT COUNT(DISTINCT student_id) AS unique_complainants
FROM complaints;
```

Query 12: Calculate Total Outstanding Fee Amount

```
SELECT SUM(amount) AS total_unpaid_dues
FROM fees
WHERE status = 'unpaid';
```

Query 13: Real-Time Bed Vacancy Audit

```
SELECT room_number, block, floor, capacity, occupied,
      (capacity - occupied) AS vacant_beds
FROM rooms
WHERE occupied < capacity
ORDER BY vacant_beds DESC;
```

5.4 Complex Subqueries & Integrity Audits

Query 14: Referential Integrity Audit — Orphaned Student Profiles

```
SELECT s.student_id, s.usn, s.course
FROM students s
LEFT JOIN users u ON s.user_id = u.id
WHERE u.id IS NULL;
```

Expected Result: Empty set. Any rows returned indicate a data integrity violation.

Query 15: Students with Fees Above System Average (Nested Subquery)

```
SELECT u.name, s.usn, s.branch, f.amount, f.due_date
FROM students s
JOIN users u ON s.user_id = u.id
JOIN fees f ON s.student_id = f.student_id
WHERE f.amount > (
   SELECT AVG(amount) FROM fees WHERE status = 'unpaid'
);
```

Query 16: Audit Private Messages — Sender & Recipient Names (Self-JOIN)

```
SELECT m.message_id,
       sender.name    AS sender_name,
       sender.role    AS sender_role,
       recipient.name AS recipient_name,
       m.content, m.is_read, m.created_at
FROM private_messages m
JOIN users sender    ON m.sender_id    = sender.id
JOIN users recipient ON m.recipient_id = recipient.id
ORDER BY m.created_at DESC;
```

Query 17: Branch-Wise Average Outstanding Fee (GROUP BY + HAVING)

```
SELECT s.branch,
       COUNT(s.student_id)          AS student_count,
       ROUND(AVG(f.amount), 2)      AS avg_outstanding_fee,
       SUM(f.amount)                AS total_dues
FROM students s
JOIN fees f ON s.student_id = f.student_id
WHERE f.status = 'unpaid'
GROUP BY s.branch
HAVING COUNT(s.student_id) > 1
ORDER BY avg_outstanding_fee DESC;
```

Query 18: Attendance Accountability — Who Marked Each Student

```
SELECT sa.date,
       student_user.name  AS student_name,
       sa.status          AS attendance_status,
       staff_user.name    AS marked_by,
       staff_user.role    AS staff_role
FROM student_attendance sa
JOIN students s          ON sa.student_id = s.student_id
JOIN users student_user  ON s.user_id     = student_user.id
JOIN users staff_user    ON sa.marked_by  = staff_user.id
ORDER BY sa.date DESC, student_user.name;
```

Chapter 6: Results and Discussion

In this chapter the results of the project are discussed. The snapshot of the project showing various modes like Administrator , Warden ,Students are showcased.

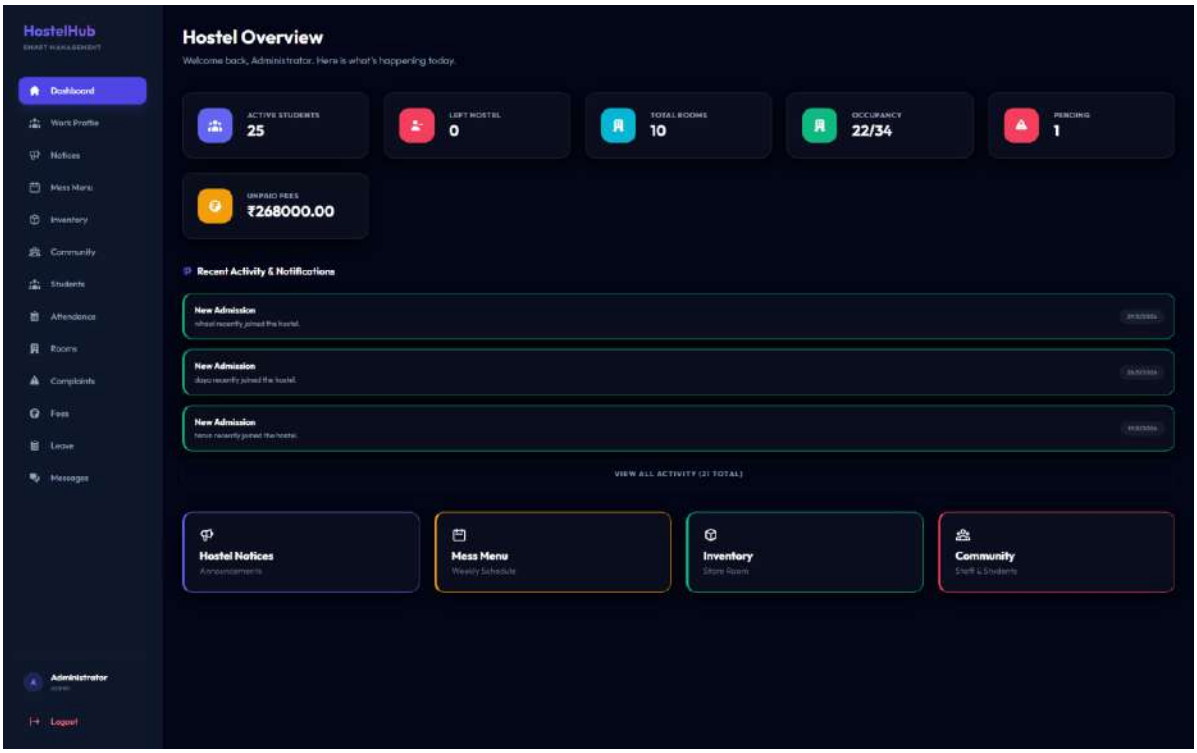


Figure 6.1

Figure 6.1 shows the Hostel Overview Dashboard page of the HostelHub system, displayed to the Warden after successful login.

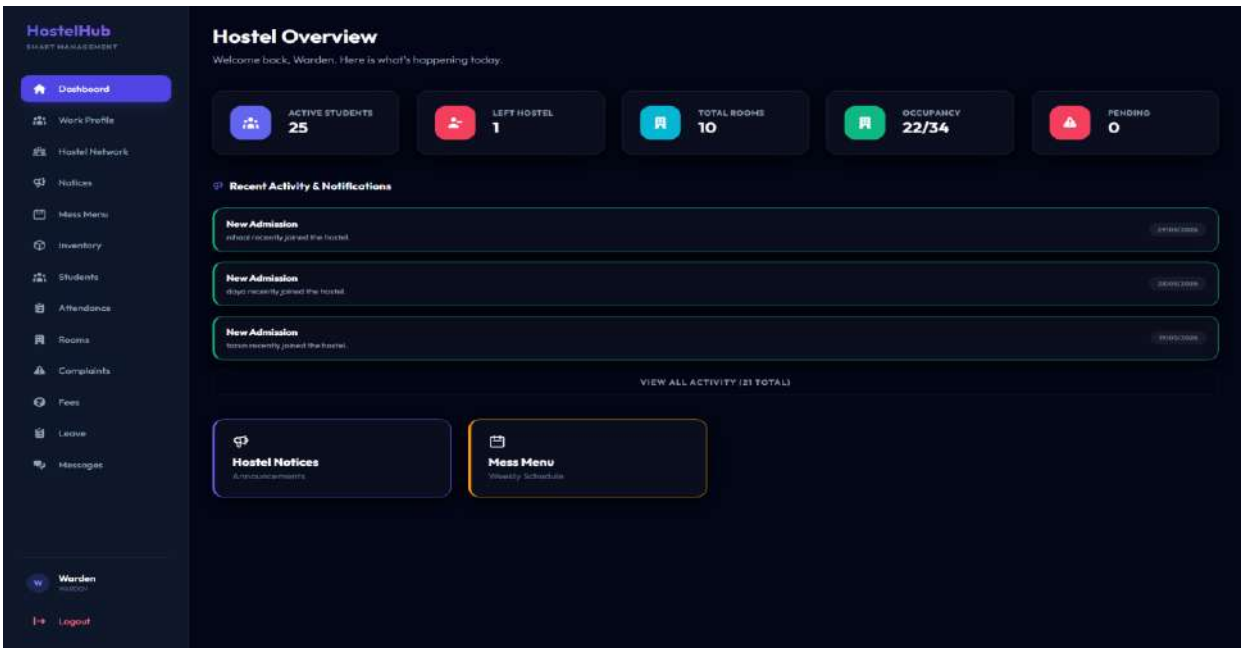


Figure 4.2

Figure 6.2 shows the Hostel Overview Dashboard page of the HostelHub system, displayed to the Warden after successful login.

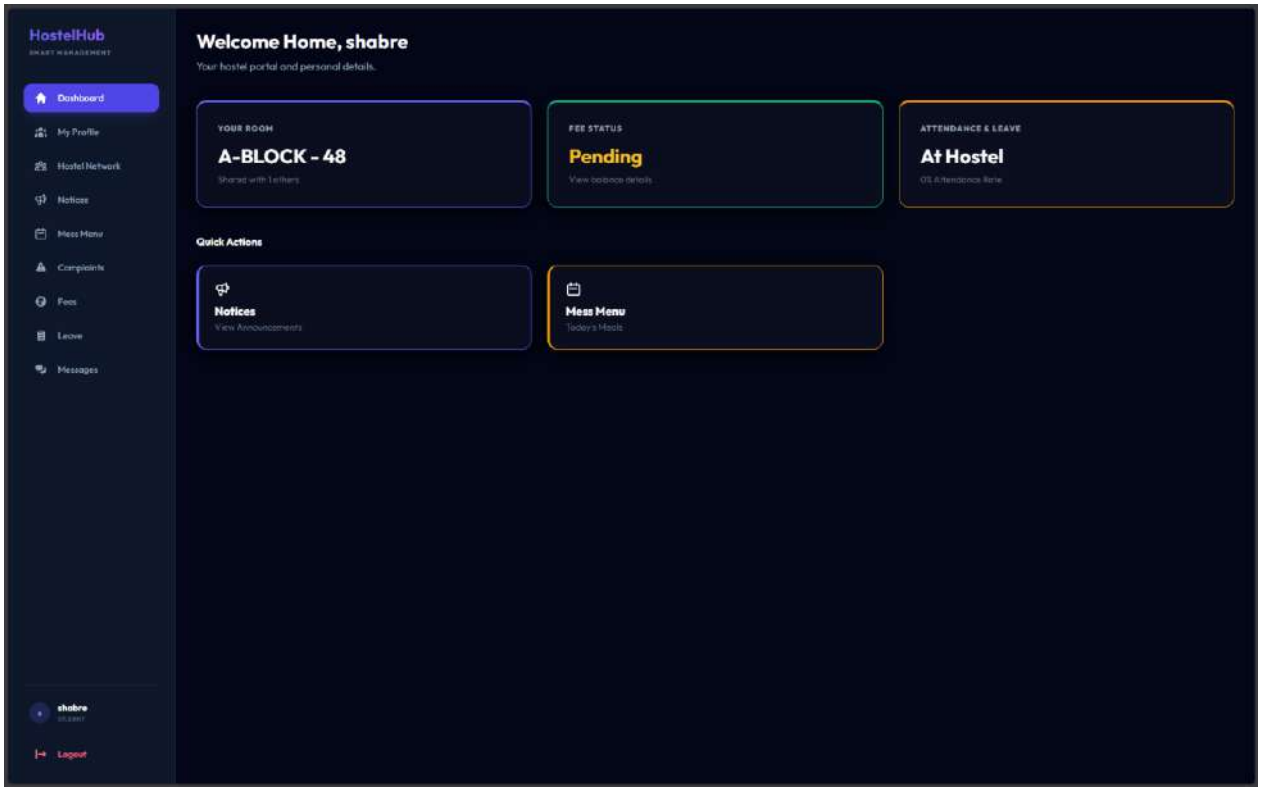


Figure 6.3

Figure 6.3 shows the Hostel Overview Dashboard page of the HostelHub system, displayed to the Warden after successful login.

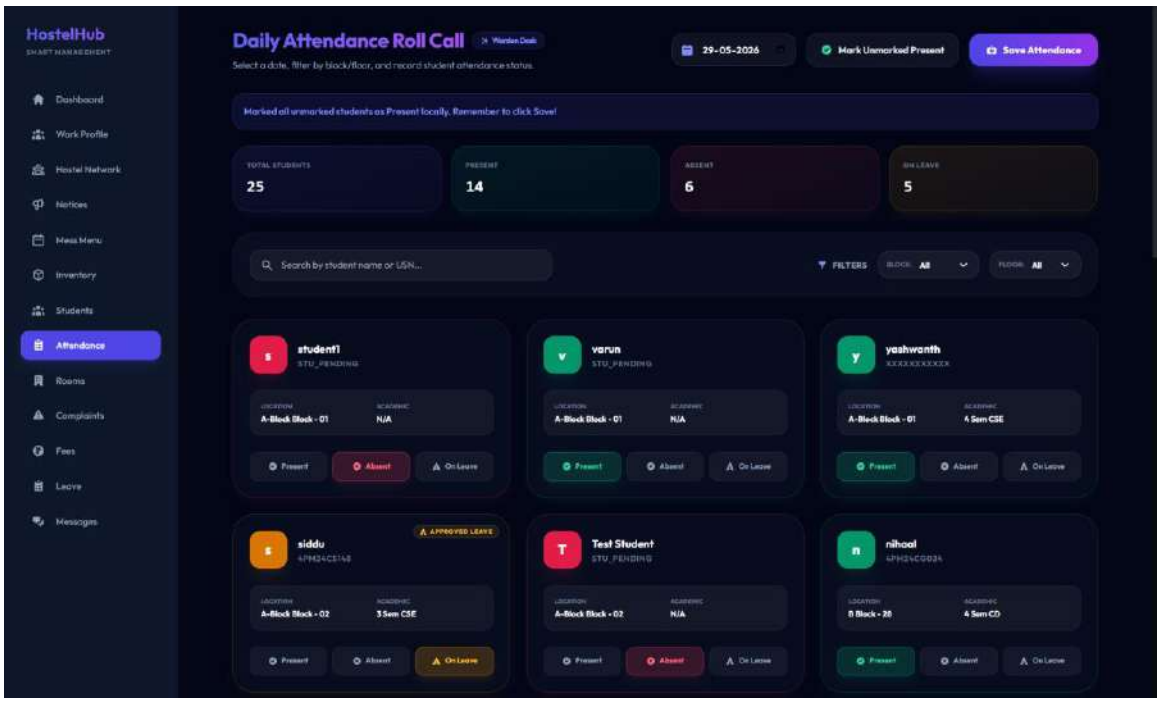


Figure 6.4

Figure 6.4 shows the Hostel Attendance Dashboard page of the system, displayed to the Warden and Administrator page only

Chapter 6: Conclusion & Future Scope

6.1 Conclusion

HostelHub successfully delivers a secure, feature-complete, and production-quality web-based hostel management system that replaces inefficient manual processes with a fully digital, role-driven workflow. The project demonstrates practical application of core computer science concepts in a real-world context:

- Database Management: 18-table relational schema with primary keys, foreign keys, ENUM constraints, composite unique keys, and DECIMAL precision types.
- Full-Stack Development: End-to-end JavaScript using React.js on the client and Node.js/Express.js on the server, connected via a REST API.
- Security Engineering: Stateless JWT-based authentication, Bcrypt password hashing, and RBAC enforced at the middleware layer.
- Software Architecture: MVC pattern with 17 route modules, 21 frontend page components, and a 3-tier client-server architecture.

Key Achievements

- Enforced room capacity constraints at the database level, eliminating the possibility of overbooking.
- Created digital pipelines for complaint resolution, leave processing, fee tracking, and payment verification.
- Implemented automated auditing for attendance records, gate safety logs, and messaging history.
- Delivered a role-adaptive dashboard serving five distinct user types from a single codebase.

6.2 Future Scope

- Biometric Attendance & Access Control: Link the gate log system to fingerprint scanners or facial recognition cameras for automated gate validation.
- IoT Utility Monitoring: Deploy smart meters and IoT sensors to track real-time electricity and water consumption per room, displayed as live graphs on the admin dashboard.
- Mobile Application: Develop a companion React Native mobile app for push notifications on leave approvals, complaint updates, and fee reminders.
- Automated Reporting & Analytics: Generate scheduled PDF reports (weekly attendance, monthly fee collection, quarterly complaint metrics) emailed automatically to administration.
- AI-Powered Chatbot: Implement a natural language chatbot for students to get instant answers on hostel rules, mess timings, fee deadlines, and complaint status.